

SPORTFIELD

Système de Gestion de Réservations de Terrains



RAPPORT TECHNIQUE DE PROJET

Application Web Django — Gestion d'Infrastructures Sportives

Année académique 2025–2026

4

Modules

6

Vues Django

7

Templates HTML

100%

Bootstrap 5

Projet	SportField — Réservation de terrains sportifs
Technologie	Python 3.14 · Django 6.0 · SQLite · Bootstrap 5
Dépôt source	GitHub (dossier compressé fourni)
Période	Avril – Mai 2026
Type document	Rapport technique de projet final
Realisé par	Mohamed Maaninou
Encadré par	M. Mohamed ghazouani



R E M E R C I E M E N T S

Il est important de souligner ici ma reconnaissance sincère à tous ceux qui ont apporté une contribution directe ou indirecte à la réussite de ce travail académique dans le cadre de l'EMSI Casablanca.

Tout d'abord, je veux exprimer ma reconnaissance au guide pédagogique de ce projet, **Monsieur Mohamed Ghazouani**, pour son conseil avisé et son appui pendant toute la durée de ce travail.

De plus, je veux exprimer ma gratitude au personnel enseignant de l'EMSI Casablanca, grâce auquel j'ai acquis par une formation de haut niveau les connaissances et compétences requises pour réaliser ce projet de développement de site Web.

Enfin, je veux remercier ma famille et mes amis pour leur soutien constant dans ce projet académique.

TABLE DES MATIÈRES

Contexte et Objectifs	4
Problématique résolue	4
Profils utilisateurs	4
Structure des Répertoires	4
Stack Technologique	4
Schéma Relationnel	5
Types de Surface Supportés	5
Vues et Endpoints	5
Algorithme Anti-Chevauchement	6
Templates HTML — Bootstrap 5	6
Interface /admin Personnalisée	6
Acteurs du Système	8
Diagramme UML	8
Classes et Attributs	10
Relations entre Classes	10
Points Forts du Projet	11
Sécurité — Points d'Attention	11

01 PRÉSENTATION DU PROJET

Contexte et Objectifs

SportField est une application web développée avec le framework Python Django dont l'objectif est de moderniser la gestion des créneaux horaires d'infrastructures sportives : complexes, parcs et gymnases. Elle s'adresse à deux types d'utilisateurs distincts et répond à un besoin concret de planification efficace des réservations sportives.

Problématique résolue

- ▶ Éliminer les conflits de double réservation sur un même terrain
- ▶ Centraliser la gestion administrative via un back-office dédié
- ▶ Offrir aux joueurs une interface intuitive de recherche et de réservation
- ▶ Assurer une vision calendaire en temps réel de l'occupation des terrains

Profils utilisateurs

Profil	Rôle	Accès principal
Gestionnaire (Admin)	Administration de la plateforme	Interface /admin Django
Joueur	Réservation de terrains	Interface web Bootstrap

02 ARCHITECTURE DU PROJET

Structure des Répertoires

Le projet respecte la convention standard de Django avec une séparation nette entre le module de configuration (sportfield/) et l'application métier (reservations/).

Répertoire / Fichier	Rôle
sportfield/settings.py	Configuration globale (DB, apps, static files)
sportfield/urls.py	Routage principal vers les sous-URLs
reservations/models.py	Définition des entités métier (ORM Django)
reservations/views.py	Logique applicative et rendu des pages
reservations/forms.py	Formulaires avec validation côté serveur
reservations/admin.py	Interface d'administration personnalisée
reservations/urls.py	Routes propres à l'application
templates/reservations/	Templates HTML (Bootstrap 5)
static/images/	Assets statiques (logo, images de fond)

Stack Technologique

Django Framework Web	SQLite Base de Données	BS5 Interface UI	Python Langage
--------------------------------	----------------------------------	----------------------------	--------------------------

03 MODÈLES DE DONNÉES

Schéma Relationnel

L'application repose sur quatre entités principales reliées entre elles par des clés étrangères. Le schéma est optimisé pour garantir l'intégrité référentielle et prévenir les conflits de réservation.

Modèle	Champs clés	Relations	Contrainte notable
Sport	nom_sport, capacite_max	→ Terrain (1-N)	Protégé à la suppression
Terrain	nom, type_surface, disponibilite	FK → Sport	4 types de surface
Joueur	nom, telephone	→ Reservation (1-N)	Identification client
Reservation	date_reservation, heure_debut, duree	FK → Terrain, Joueur	Anti-chevauchement

Types de Surface Supportés

Gazon	Synthétique	Parquet	Terre battue
-------	-------------	---------	--------------

04 LOGIQUE APPLICATIVE

Vues et Endpoints

Le fichier views.py expose six vues fonctionnelles couvrant l'intégralité du parcours utilisateur, de la page d'accueil jusqu'à la gestion du calendrier d'occupation.

Vue	URL	Méthodes	Description
home	/	GET	Page d'accueil avec hero section
terrain_list	/terrain	GET	Liste avec filtres (sport, dispo, recherche)
terrain_detail	/terrain/<pk>/	GET	Détail et réservations d'un terrain

reserver	/reserver/<id>/	GET / POST	Formulaire de réservation avec validation
calendrier	/calendrier/	GET	Réservations à venir (date ≥ aujourd'hui)
joueur_add	/joueurs/add/	GET / POST	Inscription d'un nouveau joueur

Algorithme Anti-Chevauchement

La protection contre la double réservation est implémentée à deux niveaux : au niveau du formulaire (forms.py) et au niveau du modèle (models.py). Cette double validation garantit l'intégrité même lors d'appels directs à l'ORM.

Niveau Formulaire (forms.py)

Validation avant soumission en BDD. Retourne un message d'erreur lisible à l'utilisateur indiquant l'heure du conflit.

Niveau Modèle (models.py)

Méthode clean() appelée par save(). Constitue un filet de sécurité côté ORM, indépendant de l'interface utilisée.

05 INTERFACE UTILISATEUR

Templates HTML — Bootstrap 5

L'interface est entièrement responsive grâce à Bootstrap 5.3.3 et Bootstrap Icons 1.11.3. Un template de base fournit la navbar, la gestion des messages flash et le pied de page.

Template	Fonctionnalité UI	Composants Bootstrap
base.html	Navbar verte	navbar-dark bg-success
home.html	Hero image, grille de features	container, row, col-md-4
terrain_list.html	Filtres + liste de cartes	form-control, form-select, card
terrain_detail.html	Infos terrain + tableau réservations	table, badge
reserver.html	Formulaire de réservation	form-control, DateInput, TimeInput
calendrier.html	Tableau chronologique	table-striped, order_by
joueur_add.html	Inscription joueur	form-control

06 ADMINISTRATION DJANGO

Interface /admin Personnalisée

Le fichier admin.py enregistre les quatre modèles avec des classes Admin personnalisées offrant des capacités de recherche, de filtrage et d'édition directe depuis la liste.

Modèle Admin	list_display	Filtres disponibles	Édition directe
ReservationAdmin	terrain, date, heure, durée, joueur	date_reservation, terrain	—
SportAdmin	nom_sport, capacite_max	—	—
TerrainAdmin	nom, surface, sport, dispo	sport, dispo, surface	disponibilite
JoueurAdmin	nom, telephone	—	—

La fonctionnalité list_editable sur le champ disponibilite du TerrainAdmin est particulièrement utile : l'administrateur peut basculer la disponibilité d'un terrain directement depuis la liste sans ouvrir chaque fiche.

07

DIAGRAMME DE CAS D'UTILISATION

Acteurs du Système

Le système de réservation SportField implique deux acteurs principaux. Le Joueur est l'acteur de base, l'Admin dispose de cas d'utilisation exclusifs à l'administration de la plateforme.

Acteur	Cas d'utilisation
Joueur	Rechercher terrain · Filtrer par sport · Voir disponibilité · Réserver terrain
Admin	Gérer sports · Gérer terrains · Voir calendrier · Supprimer réservations anciennes

Diagramme UML

Le diagramme ci-dessous illustre les interactions entre les acteurs et le système. La relation «include» entre Réserver terrain et Voir disponibilité traduit le fait que toute réservation nécessite obligatoirement une vérification de disponibilité préalable.

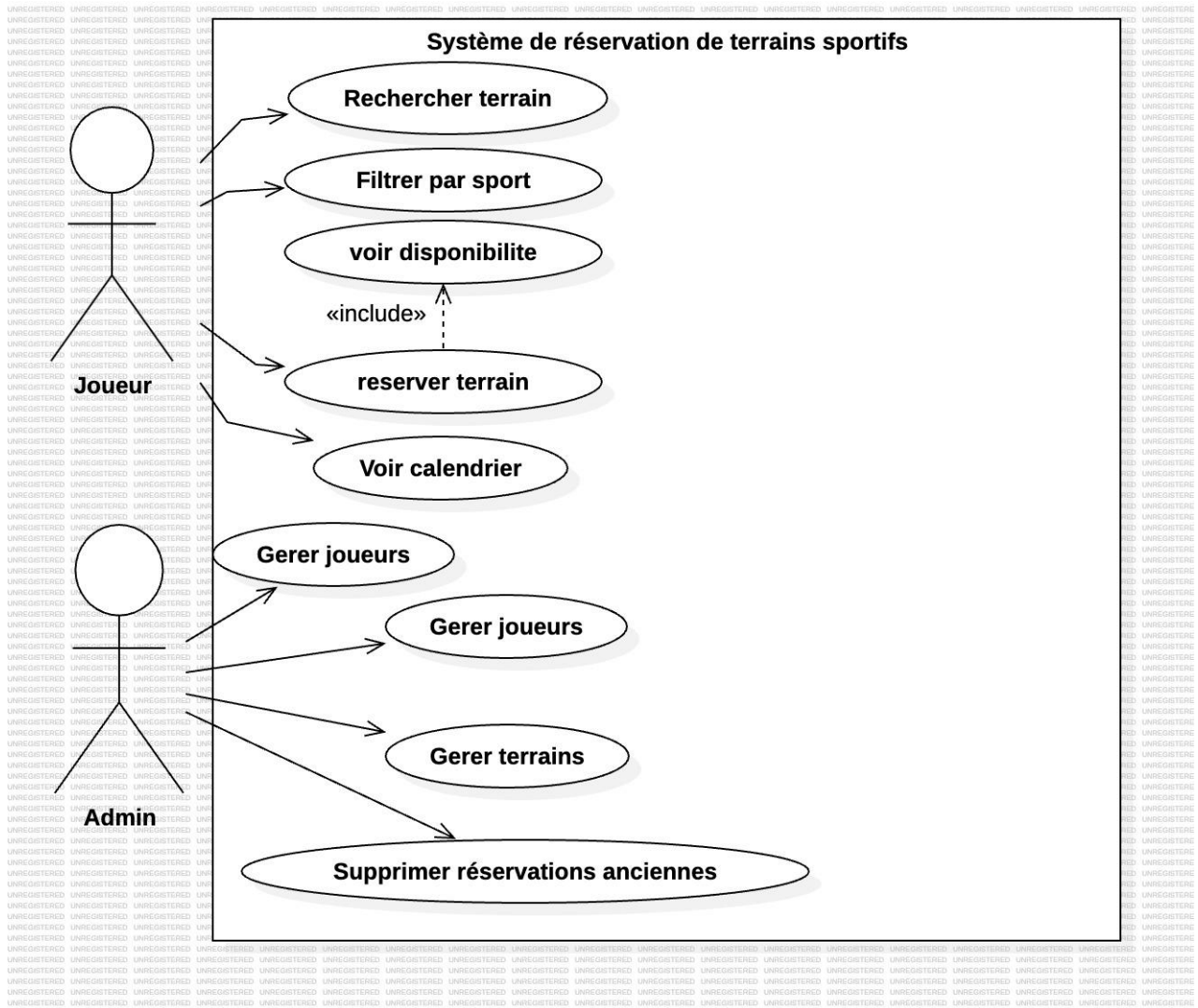
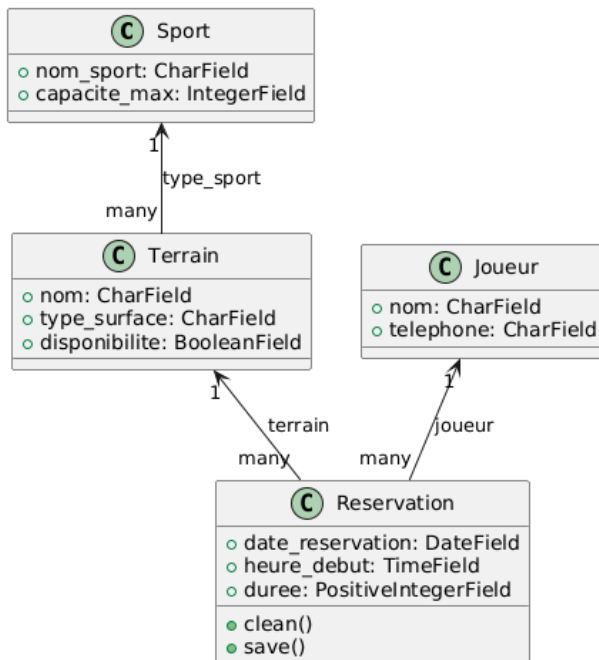


Figure 1 — Diagramme de cas d'utilisation du système SportField

08 DIAGRAMME DE CLASSES

Classes et Attributs

Le diagramme de classes reflète fidèlement les quatre modèles Django définis dans `models.py`. Chaque classe est représentée avec ses attributs typés et ses méthodes principales.



Relations entre Classes

Association	Type	Multiplicité	Description
Sport → Terrain	Composition	1 à N	Un sport peut avoir plusieurs terrains. Suppression protégée.
Terrain → Reservation	Association	1 à N	Un terrain peut avoir plusieurs réservations. Cascade.
Joueur → Reservation	Association	1 à N	Un joueur peut effectuer plusieurs réservations. Cascade.

Figure 2 — Diagramme de classes du système SportField

09

ANALYSE CRITIQUE & PERSPECTIVES

Points Forts du Projet

Forces

- ▶ Architecture MVC propre et lisible
- ▶ Double validation anti-chevauchement
- ▶ Administration riche et opérationnelle
- ▶ Design responsive Bootstrap 5
- ▶ Filtrage multi-critères efficace
- ▶ Code bien structuré

Améliorations Suggérées

- ▶ Authentification utilisateur
- ▶ Système de paiement en ligne
- ▶ Notifications email / SMS
- ▶ API REST (Django REST Framework)
- ▶ Calendrier visuel (FullCalendar.js)
- ▶ Déploiement PostgreSQL + Nginx

Sécurité — Points d'Attention

- ▶ Placer SECRET_KEY dans une variable d'environnement (.env)
- ▶ Passer DEBUG à False et configurer ALLOWED_HOSTS
- ▶ Migrer vers PostgreSQL pour un environnement de production
- ▶ Activer HTTPS (certificat SSL/TLS) et CSRF_TRUSTED_ORIGINS

10

CONCLUSION

SportField démontre une maîtrise solide du framework Django et des bonnes pratiques de développement web. Le projet couvre l'intégralité du cycle de vie d'une application : conception du modèle de données, implémentation de la logique métier, création de l'interface utilisateur et configuration de l'administration.

La double validation anti-chevauchement est particulièrement remarquable : elle témoigne d'une compréhension fine des risques d'intégrité des données et d'une approche défensive du développement. L'interface Bootstrap 5 est propre, cohérente et professionnelle.

— Fin du Rapport —